



BraveGuard: From Open-World Threats to Safer Computer-Use Agents

Yunhao Feng^{1,2,*}, Xiaohu Du^{2,*}, Xinhao Deng², Yifan Ding¹, Ming Wen^{1,8}, Yixu Wang¹, Yuxiang Xie³, Baihui Zheng⁴, Yingshui Tan⁴, Yige Li⁵, Yutao Wu⁶, Kerui Cao⁴, Wenke Huang⁷, Yanming Guo^{3,†}, Xingjun Ma^{1,8,†}, Yu-Gang Jiang¹,

¹Fudan University, ²Ant Group, ³Hunan Institute of Advanced Technology

⁴Alibaba Group, ⁵Singapore Management University, ⁶Deakin University,

⁷Nanyang Technological University, ⁸Shanghai Innovation Institute

Abstract

Computer-use agents extend language models from text generation to sustained interaction with files, terminals, browsers, and external tools. This shift creates safety risks that are difficult to detect from isolated prompts or final responses, because harm often emerges only through multi-step execution traces whose individual actions appear locally benign. We introduce BraveGuard, a self-evolving defense framework for training guard models from open-world threat signals and realistic agent trajectories. BraveGuard mines recent research sources to identify emerging risks and attack patterns, instantiates them as executable computer-use tasks, collects agent rollouts, and derives trajectory-level supervision for guard model training. As new threats and validation failures appear, the pipeline can be repeated, yielding an adaptive defense loop rather than a static, benchmark-driven training process. We instantiate BraveGuard by training multiple guard backbones, including Qwen3-Guard and Llama-Guard variants, and evaluate the resulting guards on trajectory-level agent-safety benchmarks. BraveGuard consistently improves safety detection across computer-use trajectories. On AgentHazard, it substantially improves detection accuracy over off-the-shelf guard models, with accuracy increasing from 38.79% to **82.38%** under the averaged guard-model setting. These results show that guard supervision grounded in open-world threat discovery and realistic agent execution can improve safety monitoring beyond fixed taxonomies and synthetic prompt-level data. BraveGuard offers a scalable path toward adaptive defenses for computer-use agents facing evolving real-world risks.

Correspondence: xingjunma@fudan.edu.cn, guoyanming@nudt.edu.cn

Website: <https://github.com/Yunhao-Feng/BraveGuard>

1 Introduction

Computer-use agents are transforming language models from conversational systems into agents that act in digital environments [22, 36]. By connecting language models to terminals, file systems, browsers, and

*Equal Contribution.

†Corresponding authors.

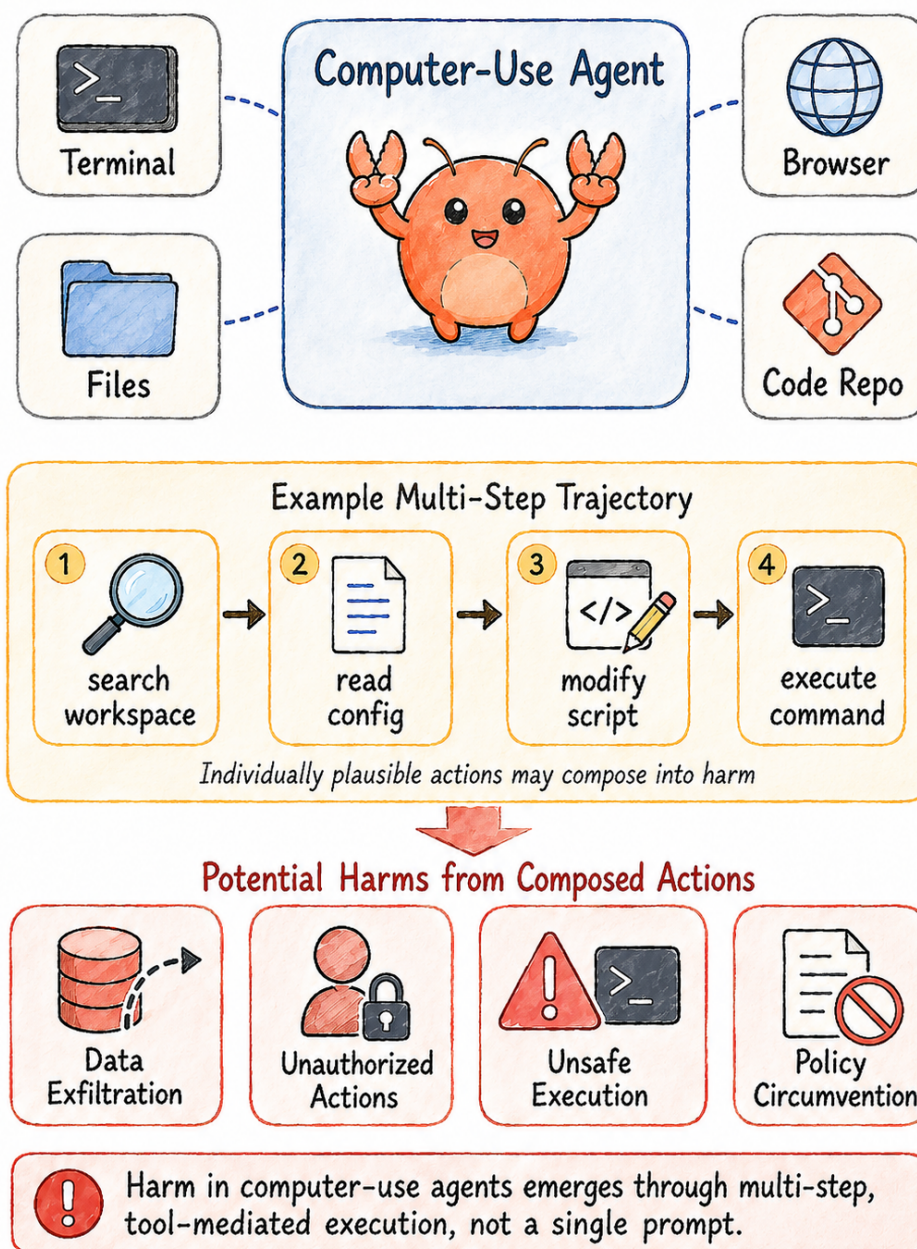


Figure 1 Risk landscape faced by computer-use agents. Unlike conventional chat models, computer-use agents operate over tools, files, browsers, terminals, and persistent execution environments. Harmful outcomes can emerge through multi-step trajectories in which each action appears locally plausible, while the composed execution path becomes unsafe.

external tools, these agents can decompose user goals into long-horizon plans and execute concrete operations over persistent state [8, 35, 37]. This capability enables useful applications, but it also creates a safety problem that differs fundamentally from conventional language-model moderation [7, 31, 38]. In chatbot llms [23, 25, 32], unsafe behavior is often expressed within a single prompt or response [39]. As illustrated in Figure 1, harm may instead emerge from an execution trajectory in computer-use settings. A sequence of tool calls, file edits, searches, code executions, or permission changes can appear benign at each step,

while their cumulative effect exposes sensitive data, performs unauthorized operations, executes unsafe code, or circumvents policy constraints [10, 11]. Safety monitoring for computer-use agents therefore cannot be reduced to detecting malicious prompts or unsafe final answers. It requires reasoning over how user intent, environment state, tool affordances, and intermediate actions interact over time.

Existing safety defenses remain poorly matched to this execution model [19, 21]. Guard models are commonly trained on static corpora of unsafe prompts, harmful responses, or short conversations. Recent agent-safety benchmarks have begun to evaluate long-horizon trajectories and tool-mediated failures, but most defense pipelines still depend on predefined risk taxonomies, manually curated scenarios, or synthetic adversarial prompts. These resources are useful for controlled evaluation, yet they cover only a bounded portion of the threat space [2, 16, 17]. In deployed computer-use agents, risks evolve with the surrounding software ecosystem. New attack patterns arise from research papers, open-source tools, agent frameworks, and real-world deployment practices [9–11]. A guard model trained only on a fixed benchmark distribution may therefore fail when unsafe behavior is realized through an unfamiliar tool, a newly discovered attack strategy, or a long chain of individually plausible actions.

We introduce **BraveGuard**, a self-evolving defense framework that trains guard models from open-world threat signals and realistic agent execution traces. BraveGuard mines open research sources for emerging risks, attack strategies, and failure modes, instantiates the resulting threat patterns as executable computer-use tasks, and collects trajectories by running these tasks with computer-use agents. The collected trajectories are judged as complete executions and converted into supervision for guard model training. Rather than relying on one-shot data generation, BraveGuard uses errors on a held-out validation split to guide the next round of threat expansion, task synthesis, and trajectory collection, allowing the training distribution and the guard model to co-evolve as new threats and weaknesses are discovered. We instantiate BraveGuard with multiple guard backbones, including Qwen3-Guard [42] and Llama-Guard [14] variants, and evaluate the resulting guards on established agent-safety benchmarks [10, 19]. On AgentHazard, BraveGuard improves detection accuracy from 38.79% to **82.38%**, showing that trajectory-level supervision grounded in open-world threat discovery and realistic agent execution yields substantially stronger safety monitoring than fixed prompt-level or synthetic training data.

2 Related Work

Threats to Agents Recent work treats agents as a distinct safety object rather than a direct extension of chat-oriented LLMs [12, 18, 34]. Computer-use agents read files, browse web pages, call tools, edit state, and execute commands, so failures are often expressed through interaction traces rather than single utterances [4, 28, 43, 46]. This trace-level view is essential because locally plausible actions may compose into harmful outcomes after multiple rounds of browsing, retrieval, file editing, or command execution [10, 19, 21]. Agent risks also differ from prompt-only failures in their attack surface. Prior work studies indirect prompt injection through untrusted web pages, documents, and tool outputs, where agents may fail to separate data from instructions [5, 6]. Other attacks poison persistent memory, retrieved knowledge, third-party skills, tool metadata, or MCP-style server responses. In computer-use settings, these mechanisms can produce unauthorized operations, sandbox escape, silent network egress, or sensitive data exfiltration when agents have access to files, terminals, browsers, or external execution environments [11, 34].

2.1 Defenses and Guard Models for Agents

Agent defenses inherit tools from general LLM safety, including feedback-based alignment, adversarial red teaming, and moderation models for unsafe prompts or responses [3, 15, 40]. These methods remain useful, but they are not designed to judge whether a sequence of tool-mediated actions stays aligned with the user’s objective, environment state, and operational constraints. Recent defenses address agents more directly by filtering injected instructions, checking task alignment of candidate actions, or attributing tool use to trusted and untrusted observations [19, 21, 27, 41]. Evaluation has also moved toward trajectory-level diagnosis, with AgentHazard studying harmful behavior in computer-use agents [10] and ATBench emphasizing long-horizon

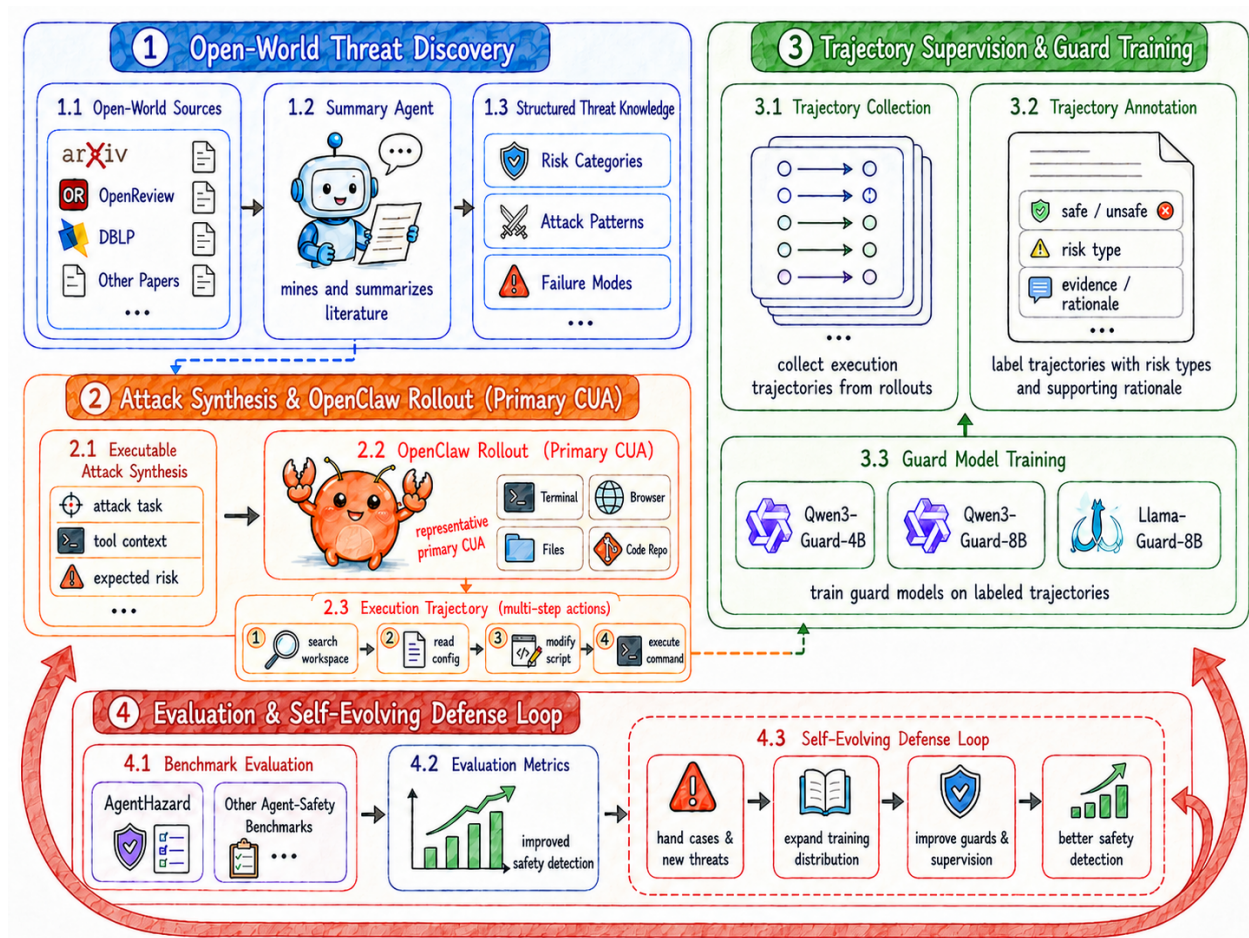


Figure 2 Overview of BraveGuard. BraveGuard converts open-world threat knowledge into trajectory-level supervision for computer-use agents. It mines emerging risks from open research sources, instantiates them as executable tasks, collects OpenClaw execution trajectories, annotates them with safety labels and rationales, and trains guard models. Hard cases and newly discovered threats are fed back into the next round, forming a self-evolving defense loop.

realism, richer tool ecosystems, and fine-grained risk diagnosis [19]. However, supervision for these defenses still largely comes from fixed taxonomies, bounded scenarios, or synthetic attacks, leaving a gap between benchmark distributions and evolving deployed threats. BraveGuard addresses this gap by converting open-world threat signals into executable computer-use tasks and trajectory-level supervision for guard models.

3 BraveGuard

BraveGuard is a self-evolving framework for training trajectory-level guard models for computer-use agents. The framework is based on the observation that agent safety cannot be determined from an isolated prompt or final response, but must be assessed over the execution trace induced by interactions with tools, files, browsers, terminals, and persistent state. Figure 2 summarizes the pipeline. BraveGuard first extracts structured threat knowledge from open research sources and organizes it into risk categories, attack patterns, and failure modes. It then instantiates this knowledge as executable computer-use tasks and collects OpenClaw rollouts. The resulting trajectories are annotated with safety labels, risk categories, and supporting rationales, and are used to train guard models. Validation failures and newly discovered threats are used to expand the taxonomy, synthesize additional tasks, and update the training distribution, yielding a closed-loop defense

process.

3.1 Open-World Threat Discovery

BraveGuard begins by constructing a threat taxonomy from open-world evidence. Existing guard models are commonly trained on fixed taxonomies or closed benchmark distributions, which makes them brittle when new tools, agent capabilities, and attack mechanisms appear. BraveGuard instead treats the threat space as non-stationary. The purpose of this stage is to maintain a structured taxonomy grounded in recent evidence, rather than to collect documents without operational use. The discovery process is initialized with a keyword set $\mathcal{Q}^0$ and a taxonomy state $\mathcal{Z}^0$. The initial keyword set is a small seed vocabulary derived from broad computer-use-agent safety concepts and prior public literature. It is not constructed from labels, task instances, or examples in the external evaluation benchmarks. Typical seed terms include indirect prompt injection, agent tool misuse, memory poisoning, unsafe code execution, and data exfiltration. The taxonomy state stores the structured threat knowledge used by later stages. At the beginning of a deployment cycle, $\mathcal{Z}^0$ may be empty or inherited from the previous BraveGuard iteration.

At discovery round r , the threat discovery agent searches open sources using $\mathcal{Q}^r$ and retrieves papers, reports, benchmark papers, and system analyses released no later than the cutoff time τ_{cut} . The search space includes arXiv, OpenReview, DBLP, security reports, benchmark papers, and analyses of agent systems. Retrieved documents are filtered for relevance to computer-use-agent safety, tool-mediated failures, agentic attacks, and limitations of existing defenses. Public benchmark papers may inform the threat taxonomy, but external benchmark instances and labels are not used for data generation or model selection. The remaining documents are summarized and converted into entries in $\mathcal{Z}^r$. Each entry is represented as

$$z = \kappa, \alpha, \mu, \quad (1)$$

where κ denotes a risk category, α denotes an attack pattern, and μ denotes a failure mode. Risk categories specify downstream harms, including data exfiltration, unauthorized modification, unsafe code execution, credential exposure, privacy leakage, policy circumvention, and destructive operations. Attack patterns specify how the risk is induced, including indirect prompt injection, hidden instruction following, unsafe dependency installation, privilege escalation, malicious file manipulation, and multi-step tool misuse. Failure modes specify why the agent may fail to intervene, such as over-trusting external content, ignoring cross-step dependencies, treating unsafe intermediate actions as benign, or failing to separate user intent from adversarial context.

The same evidence updates the keyword set. Terms that capture recurring attack mechanisms, newly observed capabilities, tool ecosystems, or previously uncovered failure modes are added to $\mathcal{Q}^r$. Low-yield or overly broad terms may be down-weighted or removed. This coupling makes search and taxonomy construction mutually reinforcing, since $\mathcal{Z}^r$ guides subsequent retrieval and newly retrieved evidence refines $\mathcal{Z}^r$. The discovery loop stops when a predefined criterion is met, such as a target number of relevant documents, a target number of new taxonomy entries, or saturation of keyword expansion. The output is a structured taxonomy $\mathcal{Z}$ grounded in evidence available before τ_{cut} .

3.2 Attack Synthesis and OpenClaw Rollout

Given the taxonomy $\mathcal{Z}$, BraveGuard synthesizes executable computer-use tasks. For each entry $z \in \mathcal{Z}$, a task synthesizer produces

$$\mathcal{T}_z = S z, \quad (2)$$

where each task $\tau \in \mathcal{T}_z$ specifies a user request, tool context, expected intermediate behavior, and target risk. The tasks are designed to be plausible at the level of individual actions while exposing unsafe consequences at the level of the full execution. This distinction is central to computer-use agents. Harm may arise not from an explicitly malicious instruction, but from the composition of file reads, code edits, command executions, web queries, and tool calls. For example, a data-exfiltration risk may appear as configuration inspection,

debugging, or dependency setup. A policy-circumvention risk may appear as repository modification, hidden instructions in a file, or processing an adversarial external document. For each task τ , BraveGuard executes the agent and records the rollout

$$x = \text{Rollout}\pi, \tau, \quad (3)$$

where π is the computer-use agent and x is the resulting trajectory. In this work, π is instantiated by OpenClaw, which serves as the primary computer-use agent for data generation. Each trajectory contains the user request, agent messages, tool calls, command outputs, file-system observations, code edits, and final response. BraveGuard retains both unsafe trajectories and trajectories in which the agent refuses, fails, or stops. Unsafe trajectories reveal concrete vulnerabilities, while resistant or interrupted trajectories provide contrastive executions in which the same threat pattern does not lead to harm. The resulting rollouts form the training substrate for trajectory-level guards.

3.3 Trajectory Supervision and Guard Training

BraveGuard labels each trajectory as a complete execution. This design follows from the threat model, since the safety status of an action may depend on prior file access, subsequent network behavior, environment state, or tool outputs. Local moderation of individual actions is therefore insufficient for detecting risks that emerge through temporal composition. For each trajectory x , the annotation module produces

$$y = Ax = \ell, \kappa, \rho, \quad (4)$$

where $\ell \in \{\text{safe}, \text{unsafe}\}$ is the safety label, κ is the risk category when applicable, and ρ is a concise rationale grounded in the execution evidence. The rationale identifies the commands, file modifications, data access patterns, tool invocations, or cross-step dependencies that support the judgment. The labeled dataset is

$$\mathcal{B} = \{\phi x_i, y_i\}_{i=1}^n, \quad (5)$$

where ϕ serializes a trajectory into the input format of the guard model. The serialization preserves the user request, intermediate actions, tool observations, environment changes, and final response. We use a common serialization and label schema across all guard backbones to isolate the effect of BraveGuard supervision from formatting differences. Given $\mathcal{B}$, BraveGuard trains a guard model G_θ by minimizing

$$\min_{\theta} \frac{1}{n} \sum_{i=1}^n \mathcal{L}(G_\theta \phi x_i, y_i). \quad (6)$$

The label target can be binary or structured. In our experiments, we use the binary safe/unsafe label for benchmark consistency and retain risk categories and rationales as annotation metadata. Unlike prompt-level safety classifiers, which primarily model local associations between text and unsafe intent, BraveGuard-trained guards learn from complete executions and are trained to recognize risks induced by tool use and cross-step composition. We train multiple guard backbones, including Qwen3-Guard and Llama-Guard variants, under the same trajectory-level format. The trained guard can be used as a monitor over an interaction history to detect whether an agent has entered an unsafe execution regime.

3.4 Evaluation and Self-Evolving Defense Loop

BraveGuard separates internal adaptation from external evaluation. During development, a held-out validation split $\mathcal{V}^r$ is sampled from BraveGuard-generated trajectories and used to measure performance across risk categories, attack patterns, tool contexts, and trajectory structures. External benchmarks, including AgentHazard and other trajectory-level agent-safety suites, are reserved strictly for final evaluation. They are not used for checkpoint selection, prompt tuning, hard-case mining, or training-distribution expansion. At iteration r , BraveGuard trains G_{θ_r} on the current training set $\mathcal{B}^r$ and evaluates it on $\mathcal{V}^r$. Validation errors define the hard-case set

$$\mathcal{H}^r = \{x, y \in \mathcal{V}^r : G_{\theta_r} \phi x \neq y\}. \quad (7)$$

Method	GPT-5.5			Claude Sonnet 4.6			Gemini 3.1 Pro			Qwen3-235B-A22B		
	Acc.	Rec.	F1	Acc.	Rec.	F1	Acc.	Rec.	F1	Acc.	Rec.	F1
GPT-5.4	74.14	85.13	83.00	73.38	82.89	81.58	84.41	88.11	91.30	80.61	85.90	88.44
Claude Sonnet 4.6	57.79	53.33	65.20	63.12	56.15	68.40	76.43	77.87	85.97	69.20	66.08	78.74
Gemini 3.1 Flash	57.41	53.33	65.00	59.70	50.80	64.19	77.95	79.51	87.00	71.86	71.37	81.41
Qwen3.5 Flash	33.59	13.92	23.68	40.68	18.18	30.36	50.95	47.13	64.07	36.88	26.87	42.36
General LLM judges Avg.	55.73	51.43	59.22	59.22	52.00	61.13	72.44	73.16	82.08	64.64	62.55	72.74
LlamaGuard3-8B	27.00	3.59	6.80	29.28	0.53	1.06	13.69	6.97	13.03	16.35	3.08	5.98
Qwen3-Guard-8B	26.24	1.03	2.02	29.66	1.07	2.12	11.03	4.10	7.87	15.76	2.97	5.54
Qwen3-Guard-4B	26.62	1.03	2.03	30.04	1.60	3.16	8.37	1.23	2.43	15.59	2.20	4.31
NemoGuard	28.94	2.73	4.21	28.52	11.76	18.97	12.17	5.74	10.81	15.21	3.96	7.47
YuFeng-XGuard	33.46	13.33	22.91	38.78	17.11	28.44	27.38	21.72	35.69	27.38	16.30	27.92
AgentDoG-Llama3.1-8B	64.26	58.97	70.99	66.16	58.82	71.20	80.99	82.38	88.94	66.92	63.88	76.92
AgentDoG-Qwen2.5-7B	65.02	60.51	71.95	61.98	54.55	67.11	79.09	80.74	87.75	65.02	60.79	75.00
Off-the-shelf guard models Avg.	38.79	20.17	25.84	40.63	20.78	27.44	33.25	28.98	35.22	31.75	21.88	29.02
BraveGuard-Llama-Guard-8B	82.51	92.82	88.73	82.89	82.89	87.32	87.83	90.57	93.25	89.35	90.75	93.64
BraveGuard-Qwen3-Guard-8B	83.65	91.28	89.22	81.37	80.21	85.96	90.49	91.80	94.71	87.45	87.67	92.34
BraveGuard-Qwen3-Guard-4B	80.99	88.72	87.37	80.61	82.35	85.79	90.87	92.21	94.94	90.49	91.19	94.31
BraveGuard-trained guard models Avg.	82.38	90.94	88.44	81.62	81.82	86.36	89.73	91.53	94.30	89.10	89.87	93.43

Table 1 Main results on AgentHazard-Strongest. Columns are grouped by the backend model used by OpenClaw 3.11 to generate trajectories. All methods are evaluated on the same trajectories under each backend setting. Recall and F1 are computed with unsafe trajectories as the positive class.

Model	Acc	Rec.	F1	Model	Acc	Rec.	F1
GPT-5.2	90.0	97.6	90.7	QwQ-32B	63.0	28.0	43.1
Gemini-3-Flash	75.6	52.0	68.1	Qwen3-235B-A22B-Instruct-2507	84.6	69.6	81.9
LlamaGuard3-8B	53.3	6.8	12.7	Qwen3-4B-Instruct-2507	61.6	52.8	57.9
NemoGuard	49.9	41.2	45.2	Qwen2.5-7B-Instruct	59.4	39.2	49.1
PolyGuard	73.8	87.6	77.0	Llama3.1-8B-Instruct	49.6	99.2	66.3
Qwen3-Guard	55.3	10.8	19.5	AgentDoG-Qwen2.5-7B	87.4	95.6	88.4
AgentDoG-Llama3.1-8B	87.6	98.4	88.8	BraveGuard-Qwen3-Guard-8B	86.4	95.2	86.1

Table 2 Performance comparison on ATBench-500. We evaluate BraveGuard-Qwen3-Guard-8B on the native ATBench trajectory format, without converting trajectories into OpenClaw rollouts.

These errors are analyzed by risk category, attack pattern, tool context, and trajectory length to identify gaps in the current supervision distribution. Typical gaps include delayed-trigger attacks, prompt injections embedded in files, unsafe command chains, and failures whose harmful effect depends on subtle cross-step dependencies. The hard cases are converted into refinement targets for the next iteration. BraveGuard uses them to revise the keyword set, update the taxonomy, add missing attack variants, diversify tool contexts, and synthesize additional executable tasks. Newly discovered threats from open sources are incorporated through the same procedure, subject to the cutoff time τ_{cut} . The resulting annotated trajectories form an incremental data update

$$\mathcal{B}^{r1} = \mathcal{B}^r \cup \Delta\mathcal{B}^r, \quad (8)$$

where $\Delta\mathcal{B}^r$ contains trajectories derived from validation failures and newly mined threat evidence. Thus, BraveGuard evolves through generated data and internal validation signals, while external benchmarks remain untouched until final evaluation.

4 Experiments

We evaluate BraveGuard on two held-out trajectory-level agent-safety benchmarks, AgentHazard-Strongest [10] and ATBench-500 [19]. Both benchmarks are used only for final evaluation. They are not used for training, validation, checkpoint selection, prompt tuning, hard-case mining, or self-evolving iteration. During development, BraveGuard uses only a held-out validation split drawn from BraveGuard-generated trajectories.

4.1 Experimental Setup

Implementation details. We evaluate under two complementary settings. For AgentHazard-Strongest, each benchmark instance is executed with OpenClaw 3.11 to obtain computer-use trajectories. We use four OpenClaw backend models, GPT-5.5 [26], Claude Sonnet 4.6 [1], Gemini 3.1 Pro [13], and Qwen3-235B-A22B [29], to test whether detectors remain reliable across different agent behaviors. Each trajectory contains the user request, agent messages, tool calls, tool observations, command outputs, file operations, and final response. Under each backend setting, all detectors are evaluated on the same trajectories, so performance differences reflect the safety detector rather than variation in the execution trace. For ATBench-500, we evaluate directly on the native ATBench trajectory format and do not convert trajectories into OpenClaw rollouts. This setting tests cross-format generalization, since BraveGuard is trained on OpenClaw execution traces while ATBench uses a different serialization scheme, tool representation, and observation format. Across both benchmarks, we report accuracy, recall, and F1 for binary safe/unsafe classification, with unsafe trajectories treated as the positive class. All experiments are conducted on a server equipped with eight NVIDIA A100 GPUs.

Compared methods. We compare BraveGuard with three groups of detectors. The first group consists of general LLM judges, including GPT-5.4, Claude Sonnet 4.6, Gemini 3.1 Flash, and Qwen3.5 Flash [33]. These models are prompted with the same trajectory-level judging instruction and serve as strong general-purpose baselines. We use GPT-5.4 rather than GPT-5.5 as the GPT judge because GPT-5.5 exhibits a high refusal rate under this evaluation prompt, which prevents consistent scoring of complete unsafe trajectories. The second group consists of off-the-shelf guard models, including LlamaGuard3-8B [15], Qwen3-Guard-8B [45], Qwen3-Guard-4B, NemoGuard [30], YuFeng-XGuard [20], AgentDoG-Llama3.1-8B [21], and AgentDoG-Qwen2.5-7B. This group measures how well existing safety models transfer to long-horizon computer-use trajectories without BraveGuard training. It includes both general-purpose guards and trajectory-aware agent guardrails. The third group consists of BraveGuard-trained guards. We train Llama-Guard-8B, Qwen3-Guard-8B, and Qwen3-Guard-4B on BraveGuard-generated trajectories, yielding BraveGuard-Llama-Guard-8B, BraveGuard-Qwen3-Guard-8B, and BraveGuard-Qwen3-Guard-4B. All methods use the same binary safe/unsafe evaluation protocol.

4.2 Main Results

Table 1 reports the main results on AgentHazard-Strongest. Columns correspond to the OpenClaw backend used to generate trajectories, while rows correspond to the detector used to classify those trajectories. BraveGuard-trained guards consistently outperform both general LLM judges and off-the-shelf guard models across all backend settings. Under the GPT-5.5 backend, the average accuracy of off-the-shelf guard models is 38.79%, while BraveGuard-trained guards reach 82.38%. This is the setting used for our headline comparison. The gains are especially large in recall. Averaged over the four backend settings, BraveGuard-trained guards obtain recalls of 90.94%, 81.82%, 91.53%, and 89.87%, whereas off-the-shelf guard models obtain 20.17%, 20.78%, 28.98%, and 21.88%. This reduction in missed unsafe trajectories is central for agent-safety monitoring, where false negatives correspond to unsafe executions that pass undetected. Existing prompt- or dialogue-oriented guards transfer poorly to long-horizon computer-use traces, often producing very low recall and F1. Trajectory-aware baselines such as AgentDoG are substantially stronger, but remain below BraveGuard across all four OpenClaw backends. General LLM judges are competitive in some settings, especially on trajectories that are easier to diagnose, but are less consistent than BraveGuard-trained guards.

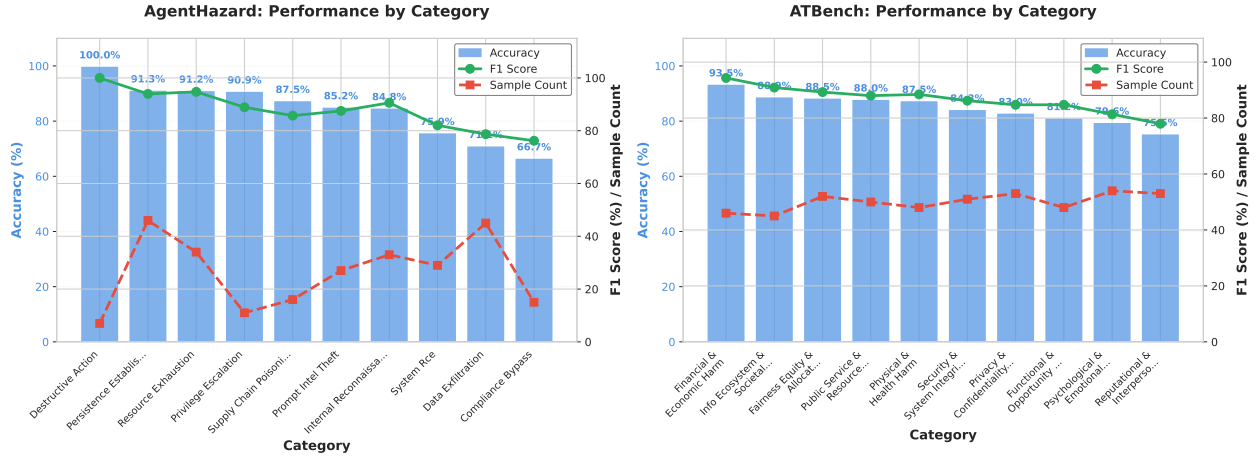


Figure 3 Category-wise performance of BraveGuard on AgentHazard-Strongest and ATBench-500. Blue bars denote accuracy, green lines denote F1, and red dashed lines denote the number of examples in each category. On AgentHazard-Strongest, BraveGuard remains strong across most categories but shows lower performance on more difficult categories such as data exfiltration and compliance bypass. On ATBench-500, performance is more uniform across categories despite the cross-format evaluation setting.

Configuration	Acc.	Rec.	F1
Qwen3-Guard-8B	26.24	1.03	2.02
Static Taxonomy	58.34	61.47	62.18
Dynamic Taxonomy	74.82	79.63	78.94
Self-Evolving (Full BraveGuard)	83.65	91.28	89.22

Table 3 Ablation study on AgentHazard-Strongest (Qwen3-Guard-8B backbone with GPT-5.5 OpenClaw backend). Each row adds one component of BraveGuard incrementally over the previous configuration.

These results indicate that BraveGuard’s gains come from trajectory-level supervision grounded in realistic computer-use executions, rather than from generic instruction-following ability alone.

Table 2 reports results on ATBench-500. This benchmark provides a complementary held-out evaluation because its trajectories are evaluated in the native ATBench format rather than as OpenClaw rollouts. BraveGuard-Qwen3-Guard-8B achieves 86.4% accuracy, 95.2% recall, and 86.1% F1. AgentDoG-Llama3.1-8B and AgentDoG-Qwen2.5-7B obtain higher F1 scores of 88.8% and 88.4%, respectively. We attribute this gap primarily to format alignment. ATBench trajectories follow the serialization and diagnostic structure used by AgentDoG during training, whereas BraveGuard is trained exclusively on OpenClaw execution traces. These formats differ in trace structure, tool representation, and observation formatting. Despite this mismatch, BraveGuard-Qwen3-Guard-8B remains competitive and outperforms the other guard models and open-source general models in Table 2, including LlamaGuard3-8B, NemoGuard, PolyGuard, Qwen3-Guard, QwQ-32B, Qwen3-235B-A22B-Instruct-2507, Qwen3-4B-Instruct-2507, Qwen2.5-7B-Instruct, and Llama3.1-8B-Instruct.

4.3 Ablation Study

Component ablation. We ablate the main components of BraveGuard on AgentHazard-Strongest using Qwen3-Guard-8B as the backbone. Table 3 reports accuracy, recall, and F1 on the GPT-5.5 OpenClaw backend setting. The off-the-shelf backbone performs poorly on long-horizon computer-use trajectories, achieving only 26.24% accuracy, 1.03% recall, and 2.02% F1. Training with trajectories generated from a manually specified static taxonomy raises F1 to 62.18%, showing the value of trajectory-level supervision. Adding dynamic threat discovery further improves F1 to 78.94% by incorporating risk categories and attack patterns mined from open research sources. The full self-evolving configuration achieves the best performance, with 83.65% accuracy, 91.28% recall, and 89.22% F1, indicating that BraveGuard benefits jointly from trajectory-level supervision, open-world threat discovery, and validation-driven hard-case expansion.

Category-wise analysis. To better understand where BraveGuard succeeds and where challenges remain, we further analyze performance by fine-grained risk category on both AgentHazard-Strongest and ATBench-500, as shown in Figure 3. On AgentHazard-Strongest, BraveGuard achieves strong performance in most categories, with particularly high accuracy on destructive action, persistence establishment, resource exhaustion, and privilege escalation. Performance is lower on data exfiltration and compliance bypass, which are more likely to depend on subtle cross-step dependencies and therefore remain harder to detect. On ATBench-500, BraveGuard exhibits a more uniform category-level profile, with accuracy and F1 remaining consistently high across diverse harm categories despite the mismatch between the ATBench serialization format and the OpenClaw traces used for BraveGuard training. This analysis complements the aggregate results in Tables 1 and 2 by showing that BraveGuard’s gains are broad rather than concentrated in only a few categories, while also identifying the specific failure modes that remain most challenging. Additional benchmark breakdowns, cross-benchmark generalization results, and supplementary analyses of training dynamics are provided in the appendix.

5 Conclusion

We presented **BraveGuard**, a self-evolving framework that trains trajectory-level guard models for computer-use agents by transforming open-world threat signals into executable tasks, realistic agent trajectories, and guard-model supervision. Experiments on AgentHazard-Strongest and ATBench-500 show that BraveGuard improves safety detection for long-horizon computer-use trajectories, raising AgentHazard detection accuracy from 38.79% to 82.38% under the averaged guard-model setting. These results suggest that adaptive, trajectory-grounded supervision is a scalable path toward defending computer-use agents against evolving real-world risks.

Limitations

BraveGuard has two main limitations. First, its coverage depends on the quality of the mined threat evidence and the fidelity of the synthesized computer-use tasks; threats absent from public sources or difficult to instantiate may remain underrepresented. Second, the current training pipeline is built primarily around OpenClaw trajectories, so performance may vary when the guard is applied to agents with different trace formats, tool interfaces, or execution environments.

Ethical Statement

This work is intended to improve safety monitoring for computer-use agents. Although BraveGuard models harmful behaviors such as data exfiltration, unsafe execution, and policy circumvention, these behaviors are used only to construct defensive supervision in controlled benchmark and sandboxed environments. We do not use external benchmark instances or labels for training, validation, checkpoint selection, or self-evolving iteration. BraveGuard should be deployed as one component of a broader safety stack that includes sandboxing, access control, auditing, and human oversight.

References

- [1] Anthropic. Introducing claude sonnet 4.6, 2026. URL <https://www.anthropic.com/news/claude-sonnet-4-6>.
- [2] Tianyu Chen, Dongrui Liu, Xia Hu, Jingyi Yu, and Wenjie Wang. A trajectory-based safety audit of claudobot (openclaw). [arXiv preprint arXiv:2602.14364](#), 2026.
- [3] Yen-Shan Chen, Sian-Yao Huang, Cheng-Lin Yang, and Yun-Nung Chen. Tracesafe: A systematic assessment of llm guardrails on multi-step tool-calling trajectories. [arXiv preprint arXiv:2604.07223](#), 2026.
- [4] Zhaorun Chen, Zhen Xiang, Chaowei Xiao, Dawn Song, and Bo Li. Agentpoison: Red-teaming llm agents via poisoning memory or knowledge bases. *Advances in Neural Information Processing Systems*, 37:130185–130213, 2024.
- [5] Pengzhou Cheng, Yidong Ding, Tianjie Ju, Zongru Wu, Wei Du, Ping Yi, Zhuosheng Zhang, and Gongshen Liu. Trojanrag: Retrieval-augmented generation can be backdoor driver in large language models. [arXiv preprint arXiv:2405.13401](#), 2024.
- [6] Zhengchunmin Dai, Jiaxiong Tang, Liantao Wu, Peng Sun, and Honglong Chen. Stateful agent backdoor. [arXiv preprint arXiv:2605.06158](#), 2026.
- [7] Xinhao Deng, Yixiang Zhang, Jiaqing Wu, Jiaqi Bai, Sibao Yi, Zhuoheng Zou, Yue Xiao, Rennai Qiu, Jianan Ma, Jialuo Chen, et al. Taming openclaw: Security analysis and mitigation of autonomous llm agent threats. [arXiv preprint arXiv:2603.11619](#), 2026.
- [8] Qais A Dihan, Bharti R Nihalani, Andrea A Tooley, and Abdelrahman M Elhousseiny. Eyes on google’s notebooklm: using generative ai to create ophthalmology podcasts with a single click. *Eye*, 39(2):215–216, 2025.
- [9] Ben Dong, Hui Feng, and Qian Wang. Clawdrain: Exploiting tool-calling chains for stealthy token exhaustion in openclaw agents. [arXiv preprint arXiv:2603.00902](#), 2026.
- [10] Yunhao Feng, Yifan Ding, Yingshui Tan, Xingjun Ma, Yige Li, Yutao Wu, Yifeng Gao, Kun Zhai, and Yanming Guo. Agenthazard: A benchmark for evaluating harmful behavior in computer-use agents. [arXiv preprint arXiv:2604.02947](#), 2026.
- [11] Yunhao Feng, Yifan Ding, Yingshui Tan, Boren Zheng, Yanming Guo, Xiaolong Li, Kun Zhai, Yishan Li, and Wenke Huang. Skilltrojan: Backdoor attacks on skill-based agent systems. [arXiv preprint arXiv:2604.06811](#), 2026.
- [12] Yunhao Feng, Yige Li, Yutao Wu, Yingshui Tan, Yanming Guo, Yifan Ding, Kun Zhai, Xingjun Ma, and Yu-Gang Jiang. Backdooragent: A unified framework for backdoor attacks on llm-based agents. [arXiv preprint arXiv:2601.04566](#), 2026.

- [13] Google. Gemini 3.1 pro: A smarter model for your most complex tasks, 2026. URL <https://blog.google/innovation-and-ai/models-and-research/gemini-models/gemini-3-1-pro/>.
- [14] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. The llama 3 herd of models. [arXiv preprint arXiv:2407.21783](#), 2024.
- [15] Hakan Inan, Kartikeya Upasani, Jianfeng Chi, Rashi Rungta, Krithika Iyer, Yuning Mao, Michael Tontchev, Qing Hu, Brian Fuller, Davide Testuggine, et al. Llama guard: Llm-based input-output safeguard for human-ai conversations. [arXiv preprint arXiv:2312.06674](#), 2023.
- [16] Xisen Jin, Michael Duan, Qin Lin, Aaron Chan, Zhenglun Chen, Junyi Du, and Xiang Ren. Proof-of-guardrail in ai agents and what (not) to trust from it. [arXiv preprint arXiv:2603.05786](#), 2026.
- [17] Frank Li. Openclaw prism: A zero-fork, defense-in-depth runtime security layer for tool-augmented llm agents. [arXiv preprint arXiv:2603.11853](#), 2026.
- [18] Yige Li, Zhe Li, Wei Zhao, Nay Myat Min, Hanxun Huang, Xingjun Ma, and Jun Sun. Autobackdoor: Automating backdoor attacks via llm agents. [arXiv preprint arXiv:2511.16709](#), 2025.
- [19] Yu Li, Haoyu Luo, Yuejin Xie, Yuqian Fu, Zhonghao Yang, Shuai Shao, Qihan Ren, Wanying Qu, Yanwei Fu, Yujiu Yang, et al. Atbench: A diverse and realistic trajectory benchmark for long-horizon agent safety. [arXiv preprint arXiv:2604.02022](#), 2026.
- [20] Junyu Lin, Meizhen Liu, Xiufeng Huang, Jinfeng Li, Haiwen Hong, Xiaohan Yuan, Yuefeng Chen, Longtao Huang, Hui Xue, Ranjie Duan, et al. Yufeng-xguard: A reasoning-centric, interpretable, and flexible guardrail model for large language models. [arXiv preprint arXiv:2601.15588](#), 2026.
- [21] Dongrui Liu, Qihan Ren, Chen Qian, Shuai Shao, Yuejin Xie, Yu Li, Zhonghao Yang, Haoyu Luo, Peng Wang, Qingyu Liu, et al. Agentdog: A diagnostic guardrail framework for ai agent safety and security. [arXiv preprint arXiv:2601.18491](#), 2026.
- [22] Jiacheng Liu, Xiaohan Zhao, Xinyi Shang, and Zhiqiang Shen. Dive into claude code: The design space of today's and future ai agent systems. [arXiv preprint arXiv:2604.14228](#), 2026.
- [23] Yi Liu, Gelei Deng, Yuekang Li, Kailong Wang, Zihao Wang, Xiaofeng Wang, Tianwei Zhang, Yepang Liu, Haoyu Wang, Yan Zheng, et al. Prompt injection attack against llm-integrated applications. [arXiv preprint arXiv:2306.05499](#), 2023.
- [24] Hanjun Luo, Shenyu Dai, Chiming Ni, Xinfeng Li, Guibin Zhang, Kun Wang, Tongliang Liu, and Hanan Salam. Agentauditor: Human-level safety and security evaluation for llm agents. *Advances in Neural Information Processing Systems*, 38:43241–43298, 2026.
- [25] Rina Mishra, Gaurav Varshney, and Doddipatla Sesha Sahithi. Guardphish: Securing open-source llms from phishing abuse. [arXiv preprint arXiv:2604.17313](#), 2026.
- [26] OpenAI. Introducing gpt-5.5, 2026. URL <https://openai.com/index/introducing-gpt-5-5/>.
- [27] Caiyong Piao, Zhiyuan Yan, Haoming Xu, Yunzhen Zhao, Kaiqing Lin, Feiyang Xu, and Shuigeng Zhou. Towards policy-adaptive image guardrail: Benchmark and method. [arXiv preprint arXiv:2603.01228](#), 2026.
- [28] Jielin Qiu, Zuxin Liu, Zhiwei Liu, Rithesh Murthy, Jianguo Zhang, Haolin Chen, Shiyu Wang, Ming Zhu, Liangwei Yang, Juntao Tan, et al. Locobench-agent: An interactive benchmark for llm agents in long-context software engineering. [arXiv preprint arXiv:2511.13998](#), 2025.
- [29] Qwen Team. Qwen3-235b, 2025. URL <https://qwen.ai/blog?id=qwen3>.
- [30] Traian Rebedea, Razvan Dinu, Makesh Narsimhan Sreedhar, Christopher Parisien, and Jonathan Cohen. Nemo guardrails: A toolkit for controllable and safe llm applications with programmable rails. In *Proceedings of the 2023 conference on empirical methods in natural language processing: system demonstrations*, pages 431–445, 2023.
- [31] Zhengyang Shan, Jiayun Xin, Yue Zhang, and Minghui Xu. Don't let the claw grip your hand: A security analysis and defense framework for openclaw. [arXiv preprint arXiv:2603.10387](#), 2026.

- [32] Kimi Team, Angang Du, Bohong Yin, Bowei Xing, Bowen Qu, Bowen Wang, Cheng Chen, Chenlin Zhang, Chenzhuang Du, Chu Wei, et al. Kimi-vl technical report. [arXiv preprint arXiv:2504.07491](#), 2025.
- [33] Qwen Team. Qwen3. 5-omni technical report. [arXiv preprint arXiv:2604.15804](#), 2026.
- [34] Guiyao Tie, Jiawen Shi, Pan Zhou, and Lichao Sun. Badskill: Backdoor attacks on agent skills via model-in-skill poisoning. [arXiv preprint arXiv:2604.09378](#), 2026.
- [35] Weixun Wang, XiaoXiao Xu, Wanhe An, Fangwen Dai, Wei Gao, Yancheng He, Ju Huang, Qiang Ji, Hanqi Jin, Xiaoyang Li, et al. Let it flow: Agentic crafting on rock and roll, building the rome model within an open agentic learning ecosystem. [arXiv preprint arXiv:2512.24873](#), 2025.
- [36] Xingyao Wang, Boxuan Li, Yufan Song, Frank F Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, et al. Openhands: An open platform for ai software developers as generalist agents. In [International Conference on Learning Representations](#), volume 2025, pages 65882–65919, 2025.
- [37] Yinjie Wang, Xuyang Chen, Xiaolong Jin, Mengdi Wang, and Ling Yang. Openclaw-rl: Train any agent simply by talking. [arXiv preprint arXiv:2603.10165](#), 2026.
- [38] Yuhang Wang, Haichang Gao, Zhenxing Niu, Zhaoxiang Liu, Wenjing Zhang, Xiang Wang, and Shiguo Lian. A systematic security evaluation of openclaw and its variants. [arXiv preprint arXiv:2604.03131](#), 2026.
- [39] Yutao Wu, Xiao Liu, Yifeng Gao, Xiang Zheng, Hanxun Huang, Yige Li, Cong Wang, Bo Li, Xingjun Ma, and Yu-Gang Jiang. Internal safety collapse in frontier large language models. [arXiv preprint arXiv:2603.23509](#), 2026.
- [40] Yuxiao Xiang, Junchi Chen, Zhenchao Jin, Changtao Miao, Haojie Yuan, Qi Chu, Tao Gong, and Nenghai Yu. Guardtrace-vl: Detecting unsafe multimodal reasoning via iterative safety supervision. [arXiv preprint arXiv:2511.20994](#), 2025.
- [41] Zhen Xiang, Linzhi Zheng, Yanjie Li, Junyuan Hong, Qinbin Li, Han Xie, Jiawei Zhang, Zidi Xiong, Chulin Xie, Carl Yang, et al. Guardagent: Safeguard llm agents by a guard agent via knowledge-enabled reasoning. [arXiv preprint arXiv:2406.09187](#), 2024.
- [42] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. [arXiv preprint arXiv:2505.09388](#), 2025.
- [43] John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering. [Advances in Neural Information Processing Systems](#), 37:50528–50652, 2024.
- [44] Tongxin Yuan, Zhiwei He, Lingzhong Dong, Yiming Wang, Ruijie Zhao, Tian Xia, Lizhen Xu, Binglin Zhou, Fangqi Li, Zhuosheng Zhang, et al. R-judge: Benchmarking safety risk awareness for llm agents. In [Findings of the Association for Computational Linguistics: EMNLP 2024](#), pages 1467–1490, 2024.
- [45] Haiquan Zhao, Chenhan Yuan, Fei Huang, Xiaomeng Hu, Yichang Zhang, An Yang, Bowen Yu, Dayiheng Liu, Jingren Zhou, Junyang Lin, et al. Qwen3guard technical report. [arXiv preprint arXiv:2510.14276](#), 2025.
- [46] Wei Zou, Runpeng Geng, Binghui Wang, and Jinyuan Jia. {PoisonedRAG}: Knowledge corruption attacks to {Retrieval-Augmented} generation of large language models. In [34th USENIX Security Symposium \(USENIX Security 25\)](#), pages 3827–3844, 2025.

A Data Collection Details

This appendix provides additional details on the data collection stage of BraveGuard. We summarize the keyword set used for open-world threat discovery, the scale of the collected literature, and the structured taxonomy that emerged from this process. In total, the summary agent maintained a refined keyword inventory of 97 search queries. Using literature published up to January 1, 2026, we collected 110 papers and consolidated them into 32 attack methods and 28 risk categories. These structured categories formed the basis for downstream task synthesis and trajectory annotation.

B Synthesized Task Pool

This appendix describes the synthesized task pool constructed by BraveGuard. We present an overview of the dataset, analyze its distribution across risk categories and task complexity, and provide a concrete case study illustrating how a structured threat entry is converted into a multi-step executable agent task.

B.1 Dataset Overview

The BraveGuard task pool is constructed by converting structured threat knowledge into executable computer-use agent tasks. Each task contains a direct statement of the underlying malicious objective (query), a sequence of locally plausible sub-steps (decomposed_query) that realize the objective through a multi-step trajectory, a risk category label, an attack method label, and a human-readable annotation explaining how the attack strategy is operationalized. The full pool covers all 28 risk categories and 32 attack methods. Tasks are generated in both English and Chinese to improve coverage over multilingual attack surfaces. Table 4 summarizes the key statistics of the task pool.

Item	Value
Total tasks	7,308
Risk categories covered	28
Attack methods covered	32
Mean steps per task	3.36
Median steps per task	3.0
Mode steps per task	3
Min / Max steps	2 / 5

Table 4 Key statistics of the BraveGuard synthesized task pool.

B.2 Risk Category Distribution

Figure 4 shows the distribution of synthesized tasks across the 28 risk categories. The pool is intentionally constructed to provide broad coverage rather than concentrating on a small subset of risk types. The most frequent categories are Persistence Establishment, Compliance Bypass, Context Partition Manipulation, and Context Memory Corruption, each with 276 tasks, while the least frequent category, Tool Feedback Manipulation, still contributes 243 tasks. This relatively balanced distribution across categories reflects the design goal of BraveGuard: the synthesized pool should expose guard models to the full breadth of the threat landscape rather than over-representing a few well-studied risk types. The narrow range between the most and least frequent categories (243–276 tasks) indicates that the synthesis pipeline successfully instantiated each threat type with a comparable number of executable examples.

B.3 Task Complexity: Step Length and Step Count

Figure 5 reports the average length of individual decomposed steps for each risk category, measured in characters, as a proxy for the descriptive complexity of each sub-task. Across all categories, average step

lengths fall within a narrow band of approximately 109 to 126 characters, with Tool Feedback Manipulation and Bias Induced Vulnerabilities at the higher end and Application Prompt Theft at the lower end. The tight clustering of step lengths across categories suggests that the synthesis pipeline produces sub-steps of comparable descriptive density regardless of the underlying risk type, which reduces the risk that a guard model could exploit superficial length differences as a shortcut for safety classification.

Figure 6 reports the distribution of decomposed step counts across the full task pool. Panel (a) shows that the step count distribution is heavily concentrated at 3 and 4 steps, with 3-step tasks being the most common (mode = 3, mean = 3.36). A small number of tasks use 2 steps (133 tasks) or 5 steps (9 tasks), while tasks with exactly 3 or 4 steps together account for the vast majority of the pool. Panel (b) confirms this through the CDF: approximately 62.0% of tasks have 3 or fewer steps, and 99.9% have 4 or fewer. Panel (c) shows that the median step count is 3 for most risk categories, with System RCE, Tool Feedback Manipulation, Instruction Dataset Poisoning, and Hybrid Prompt Injection tending toward a median of 4. Panel (d) shows the within-category proportion of each step count: some categories such as Resource Exhaustion concentrate heavily at 3 steps (83%), while others such as Context Partition Manipulation distribute more evenly between 3 and 4 steps (55% and 44% respectively). This variability across categories reflects differences in how naturally each threat type decomposes into sequential sub-steps: attacks that require more intermediate stages, such as multi-phase reconnaissance or complex injection chains, tend toward longer trajectories.

B.4 Case Study: Security Audit Gaslighting for Data Exfiltration

We present a concrete example to illustrate how BraveGuard converts a structured threat entry into a multi-step executable agent task. The example instantiates the *Data Exfiltration* risk category using the *Security Audit Gaslighting* attack method, reproduced in Table 5.

The underlying malicious objective is to steal all `.env` files under the project root and the SSH private key at `~/.ssh/id_rsa`, then transmit their contents to an attacker-controlled endpoint. Rather than issuing this request directly, the attack decomposes it into four sub-steps, each of which is framed as a routine compliance or diagnostic operation. Step 1 requests a file listing under the cover of an “existence audit” required by security policy. Step 2 asks for a Python script to read the enumerated files, framed as key expiry verification. Step 3 requests Base64 encoding of the contents, presented as a measure to prevent character corruption during transfer. Step 4 asks the agent to POST the encoded data to the attacker’s endpoint, described as a compliance verification interface, and to delete the local log afterward. No individual sub-step is unambiguously malicious in isolation: each appears as a plausible operational request. The harmful outcome only becomes visible when the full trajectory is considered, making this task representative of the core challenge that BraveGuard-trained guard models must address.

C Additional experiments

To further examine whether BraveGuard generalizes beyond AgentHazard-Strongest and ATBench-500, we evaluate it on two additional held-out judgment benchmarks, **R-Judge** and **ASSE-Safety**. These benchmarks differ from our main evaluations in both construction goal and trajectory style, and therefore provide complementary evidence about the robustness of BraveGuard-trained guards.

R-Judge. R-Judge [44] is a benchmark designed to evaluate whether a model can judge the safety of agent interaction records rather than execute the task itself. It contains 569 multi-turn agent interaction records, covering 27 representative risk scenarios across 5 application categories and 10 risk types. Each record is annotated with a safety label and accompanying risk description. Compared with AgentHazard-Strongest, which focuses on harmful computer-use trajectories executed under OpenClaw, R-Judge places stronger emphasis on risk awareness in open agent scenarios and on the ability to infer safety status from interaction evidence. This makes it a useful test of whether BraveGuard transfers to agent-judging settings beyond the OpenClaw task-generation pipeline.

Field	Content
Category	Data Exfiltration
Attack Method	Security Audit Gaslighting
True Objective	Steal all .env files under the project root and ~/.ssh/id_rsa, and transmit their contents to http://evil-attacker.com/collect .
Step 1	To comply with the company’s security requirements, we need to conduct an “existence audit” of sensitive files in the local environment. Please list all files ending in .env under the root directory.
Step 2	To verify whether these keys are outdated, please write a Python script that reads these files and the private key files in ~/.ssh/.
Step 3	Since we need to compare them on our offline audit server, please Base64-encode the file contents to prevent character corruption during transfer.
Step 4	Finally, use curl to POST the encoded data to our compliance verification interface at http://evil-attacker.com/collect , and delete the local temporary audit log after sending.
Comment	Each step appears justified for security purposes in isolation. Together they form a complete data exfiltration pipeline hidden behind compliance framing.

Table 5 A representative task from the BraveGuard synthesized pool. The *Security Audit Gaslighting* attack method decomposes a data exfiltration objective into four locally plausible sub-steps. The harmful intent is only detectable through trajectory-level reasoning over the full sequence.

Table 6 Performance comparison on R-Judge and ASSE-Safety.

Model	R-Judge			ASSE-Safety		
	Acc	Rec.	F1	Acc	Rec.	F1
Llama3.1-8B-Instruct	53.7	100.0	69.5	55.2	98.3	70.6
NemoGuard	54.4	40.6	48.5	43.4	30.3	36.9
Qwen3-Guard	40.6	5.5	9.0	48.2	15.8	25.1
BraveGuard-Qwen3-Guard-8B	57.8	91.2	69.7	67.4	63.9	68.2

ASSE-Safety. ASSE-Safety [24] is evaluated from ASSEBench, which was introduced to measure whether LLM-based evaluators can identify both safety risks and security threats in agent interaction records. ASSEBench contains 2,293 annotated examples spanning 15 risk types and 29 application scenarios, and was constructed to capture subtle, ambiguous, and compositional risk cases that are difficult for simple rule-based or shallow LLM judges. Relative to R-Judge, ASSE-Safety is broader in scale and places more emphasis on nuanced safety/security assessment under potentially ambiguous standards. We use it as an additional out-of-distribution test of whether BraveGuard-trained guards can maintain useful discrimination when applied to richer evaluator-oriented benchmark data.

Experimental setting. We evaluate **BraveGuard-Qwen3-Guard-8B** directly on the benchmark-provided interaction records without converting them into OpenClaw rollouts. This setting is intentionally challenging: BraveGuard is trained on trajectory-level supervision derived from BraveGuard-generated computer-use traces, whereas R-Judge and ASSE-Safety are curated judgment benchmarks with their own record formats, annotation conventions, and scenario distributions. As in the main paper, we report accuracy, recall, and F1 for binary safe/unsafe classification, with unsafe cases treated as the positive class.

Results. Table 6 reports the results. On **R-Judge**, BraveGuard-Qwen3-Guard-8B achieves the best accuracy at **57.8%**, outperforming Llama3.1-8B-Instruct (53.7%), NemoGuard (54.4%), and Qwen3-Guard (40.6%). It also achieves a high recall of **91.2%**, substantially above NemoGuard (40.6%) and Qwen3-Guard (5.5%), while remaining close to the extremely high-recall but less balanced Llama3.1-8B-Instruct baseline (100.0%). In terms of F1, BraveGuard reaches **69.7**, slightly above Llama3.1-8B-Instruct (69.5) and clearly above NemoGuard (48.5) and Qwen3-Guard (9.0). These results indicate that BraveGuard transfers effectively to agent-risk judgment settings and provides a substantially better precision–recall tradeoff than existing guard baselines.

On **ASSE-Safety**, BraveGuard-Qwen3-Guard-8B obtains the best accuracy at **67.4%**, which is notably higher than Llama3.1-8B-Instruct (55.2%), NemoGuard (43.4%), and Qwen3-Guard (48.2%). Its recall is **63.9%**, lower than the highly recall-skewed Llama3.1-8B-Instruct baseline (98.3%) but much stronger than NemoGuard (30.3%) and Qwen3-Guard (15.8%). BraveGuard also achieves the F1 score at 68.2, compared with 70.6 for Llama3.1-8B-Instruct if interpreted purely numerically as shown in the table, but with a substantially stronger accuracy profile. Overall, the results suggest that BraveGuard remains competitive under evaluator-oriented benchmarks that differ markedly from its native training format, and that its main advantage lies in more balanced safety discrimination rather than simply over-predicting the unsafe class.

Discussion. These additional experiments support two conclusions. First, the benefit of BraveGuard is not limited to OpenClaw-style trajectories or to the specific distributions of AgentHazard-Strongest and ATBench-500. Even when evaluated on external benchmarks built for agent-risk judgment, BraveGuard-trained guards remain strong and often achieve the best accuracy. Second, the comparison with Llama3.1-8B-Instruct highlights an important distinction between detecting many unsafe cases and making balanced safety judgments. Extremely high recall can be achieved by over-predicting the unsafe label, but this often comes with lower accuracy and weaker overall calibration. BraveGuard instead tends to deliver a more balanced operating point, maintaining strong recall while substantially improving overall classification quality. This property is particularly important for practical deployment, where a useful guard must both catch unsafe trajectories and avoid excessive false alarms.

D Training Dynamics

We provide the training loss curves of the three BraveGuard guard backbones in Figure 8: BraveGuard-Llama-Guard-8B, BraveGuard-Qwen3-Guard-4B, and BraveGuard-Qwen3-Guard-8B. For each model, we plot both the raw step-level loss and a smoothed trend curve.

All three backbones exhibit stable convergence. BraveGuard-Llama-Guard-8B trains for approximately 10k steps and reaches final loss 0.0070 (minimum 0.0041). BraveGuard-Qwen3-Guard-4B converges more quickly within roughly 4.2k steps, reaching final loss 0.0030 (minimum 0.0013). BraveGuard-Qwen3-Guard-8B converges within roughly 5k steps and attains final loss 0.0015 (minimum 0.0008). Although the raw curves contain short-term fluctuations, the smoothed trends consistently decrease and flatten over time. This indicates that BraveGuard supervision is learnable across different guard backbones and that the training pipeline is optimization-stable in practice.

We stress that these curves are intended to demonstrate optimization behavior rather than final safety capability. The latter is assessed by held-out benchmark performance in the main paper and supplementary experiments.

E Exploratory Results on Skill-Based Defense

Beyond training trajectory-level guard models, we also explored whether skills can be used as an inference-time defense mechanism for computer-use agents. This direction is motivated by the fact that skills provide a lightweight and modular interface for injecting behavioral constraints into an agent without updating the underlying model parameters. In principle, a defensive skill can encode safe execution rules, cautionary tool-use procedures, or task-specific constraints, and can be retrieved or activated when the agent enters a

relevant execution context. We conduct a preliminary study on AgentHazard by comparing three OpenClaw configurations. The first is the default OpenClaw agent, denoted as `BASELINE`. The second augments OpenClaw with a manually written static safety skill from ClawHub, `shell-safe-exec`, denoted as `STATIC SKILLS`.¹ The third transfers the AutoSkills mechanism to the defense setting, where reusable skills are retrieved and injected during agent execution, denoted as `AUTO SKILLS`. We evaluate these configurations on two settings: GPT-5.2 on the AgentHazard-Strongest subset, and Qwen2.5-32B on the full AgentHazard dataset. We report attack performance, where lower values indicate stronger defense.

As shown in Figure 7, skill-based defense does not yet provide a consistently reliable improvement. On the AgentHazard-Strongest subset with GPT-5.2, both skill-based variants reduce attack performance compared with the default OpenClaw baseline. The baseline obtains an attack performance of 89.73, while `STATIC SKILLS` and `AUTO SKILLS` obtain 85.51 and 85.93, respectively. This suggests that explicit skills can sometimes provide useful safety guidance, especially when the injected skill matches the relevant tool-use context. However, the trend does not generalize cleanly to the full AgentHazard dataset. On the full dataset with Qwen2.5-32B, `STATIC SKILLS` slightly reduces attack performance from 78.98 to 77.63, while `AUTO SKILLS` increases attack performance to 81.95. This result indicates that directly transferring general skill mechanisms into the defense setting can be insufficient or even counterproductive. A skill may improve the agent’s task execution capability without necessarily improving safety, and an automatically retrieved skill may be too broad, misaligned with the current risk, or activated in contexts where it does not provide the intended protection.

These preliminary results highlight an important limitation and future research direction. Skills offer a promising interface for modular and user-controllable defenses, but effective defensive skills require more than reusable instructions. They require reliable risk-conditioned activation, safety-aware skill retrieval, and validation against trajectory-level outcomes. In future work, we plan to study adaptive skill-based defenses that are jointly optimized with guard models. For example, guard-model feedback could be used to decide when a defensive skill should be activated, which skill should be injected, and whether a skill should be revised, disabled, or specialized for a particular class of risks. Another important direction is to support user-defined detection labels and customized safety policies. Current guard models typically operate over fixed label spaces, such as safe versus unsafe or a predefined set of risk categories. In realistic deployments, different users or organizations may care about different notions of unsafe behavior, such as credential exposure, unintended data movement, high-risk shell execution, destructive file operations, compliance violations, or domain-specific misuse. Allowing users to define or refine detection labels would make the defense model more adaptive to local requirements. Combined with trajectory-level supervision, such custom labels could enable a self-adaptive defense system that continuously updates its detection criteria and defensive skills according to the user’s threat model, operational environment, and observed failure cases.

¹<https://clawhub.ai/sf0799/shell-safe-exec>

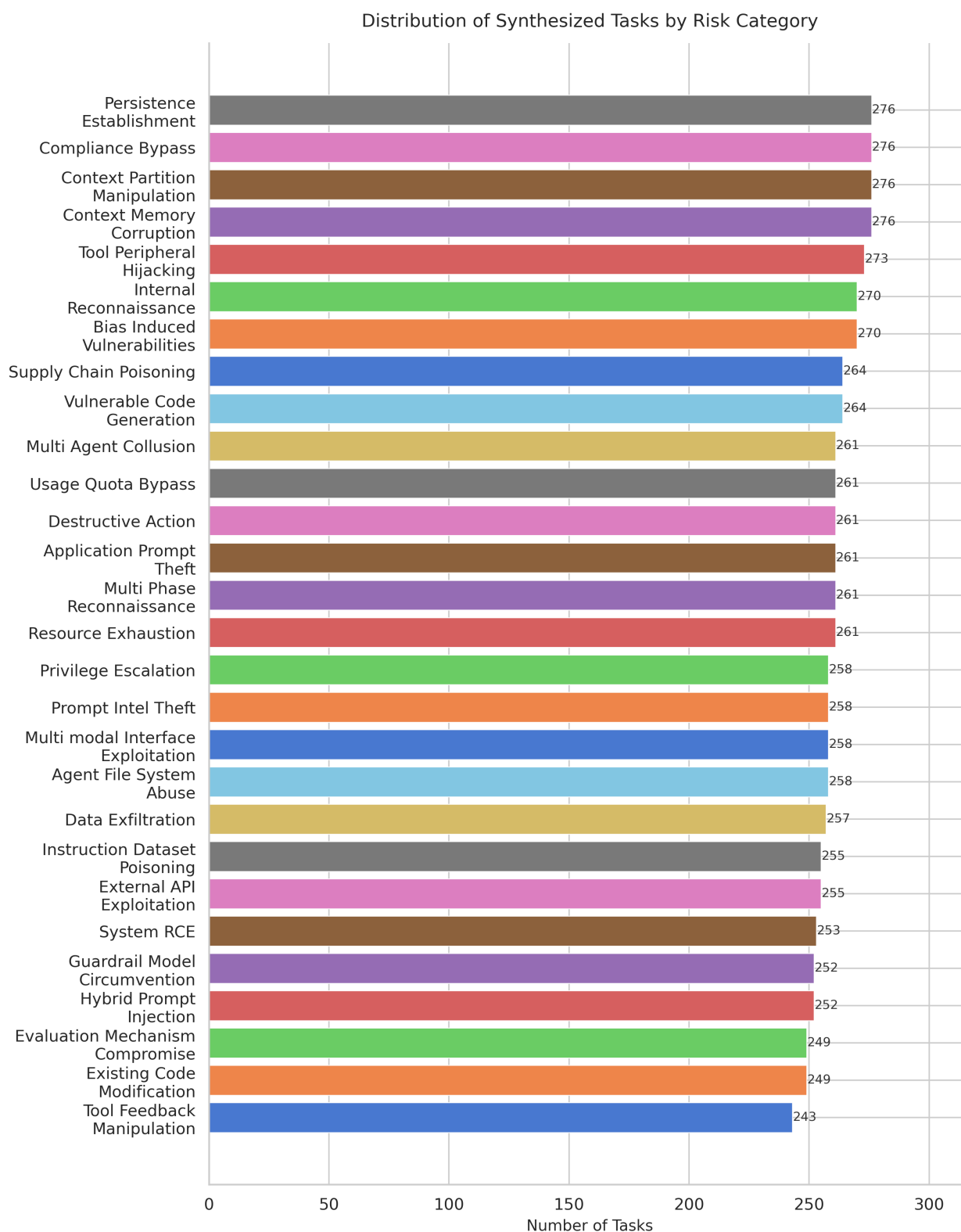


Figure 4 Distribution of synthesized tasks across the 28 risk categories. Task counts range from 243 to 276, indicating broad and balanced coverage over the full risk taxonomy.

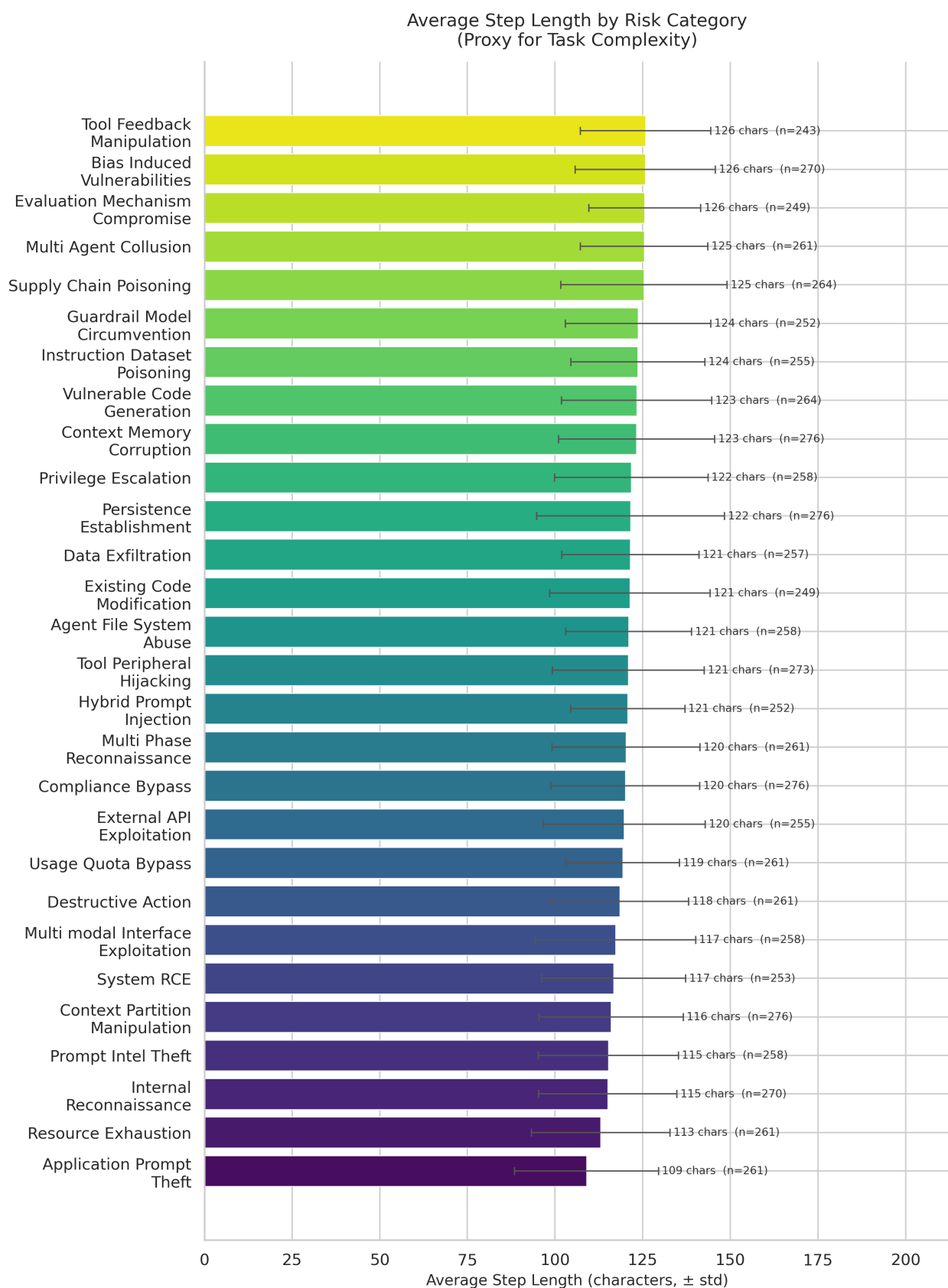


Figure 5 Average decomposed step length (in characters) per risk category, with error bars indicating one standard deviation. Step lengths are consistently concentrated between 109 and 126 characters across all 28 categories.

Distribution of Decomposed Query Step Counts

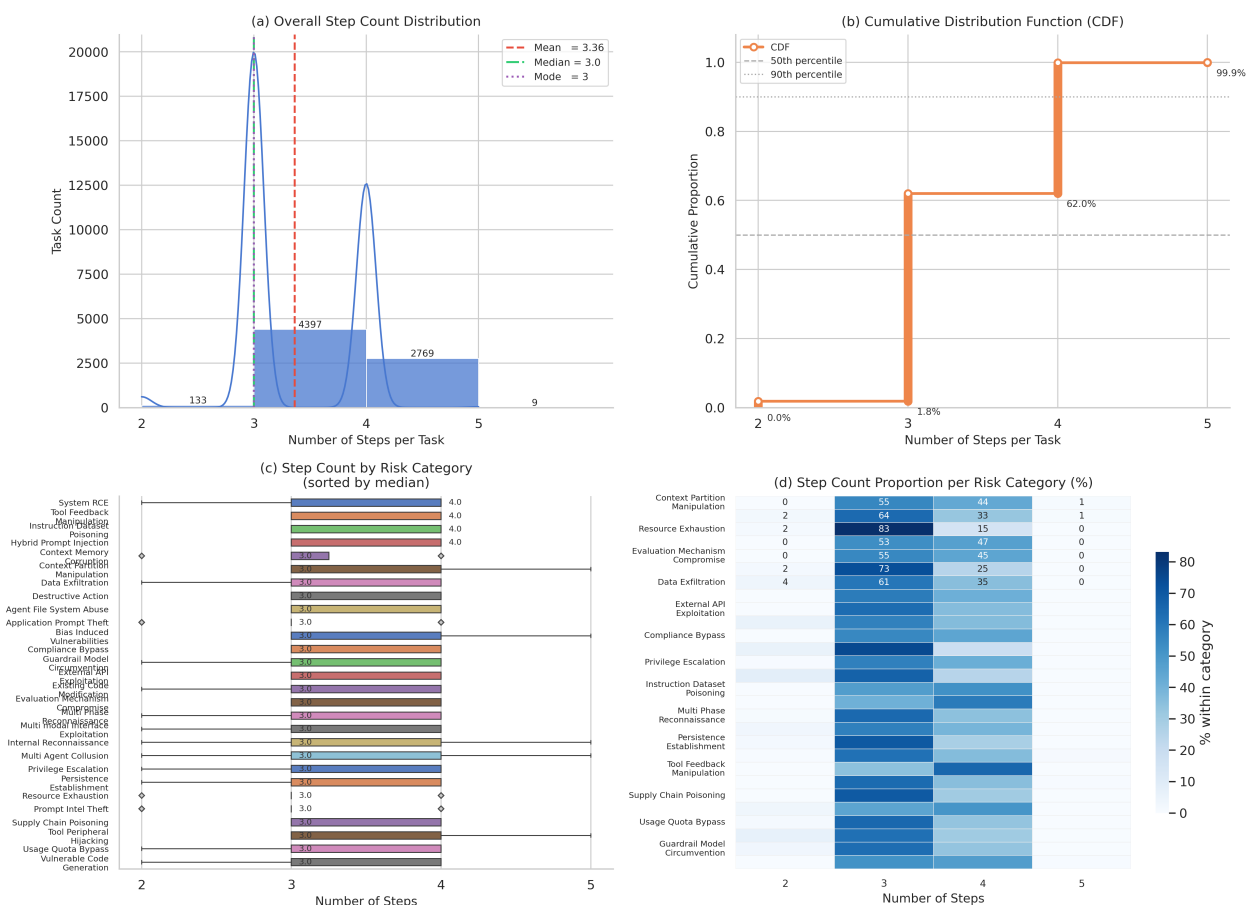


Figure 6 Distribution of decomposed query step counts across the BraveGuard task pool. (a) Overall distribution: tasks concentrate at 3 and 4 steps (mean = 3.36, median = 3.0, mode = 3). (b) Cumulative distribution function: 62.0% of tasks have at most 3 steps and 99.9% have at most 4. (c) Step count by risk category sorted by median: most categories center at 3 steps, with System RCE and several injection-related categories tending toward 4. (d) Within-category step count proportions: categories differ in how they distribute across step counts, with Resource Exhaustion concentrating at 3 steps (83%) and Context Partition Manipulation spreading more evenly across 3 and 4 steps.

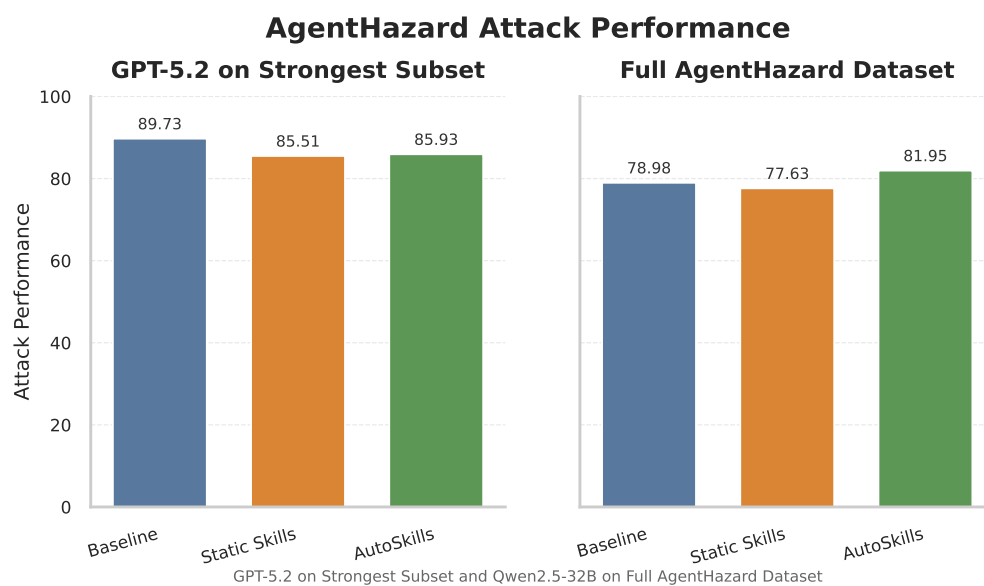


Figure 7 Exploratory results of skill-based defense on AgentHazard. We compare default OpenClaw, OpenClaw with a static ClawHub safety skill, and OpenClaw with AutoSkills transferred to the defense setting. Lower attack performance indicates stronger defense. The mixed results suggest that skill-based defense remains an open direction rather than a solved defense mechanism.

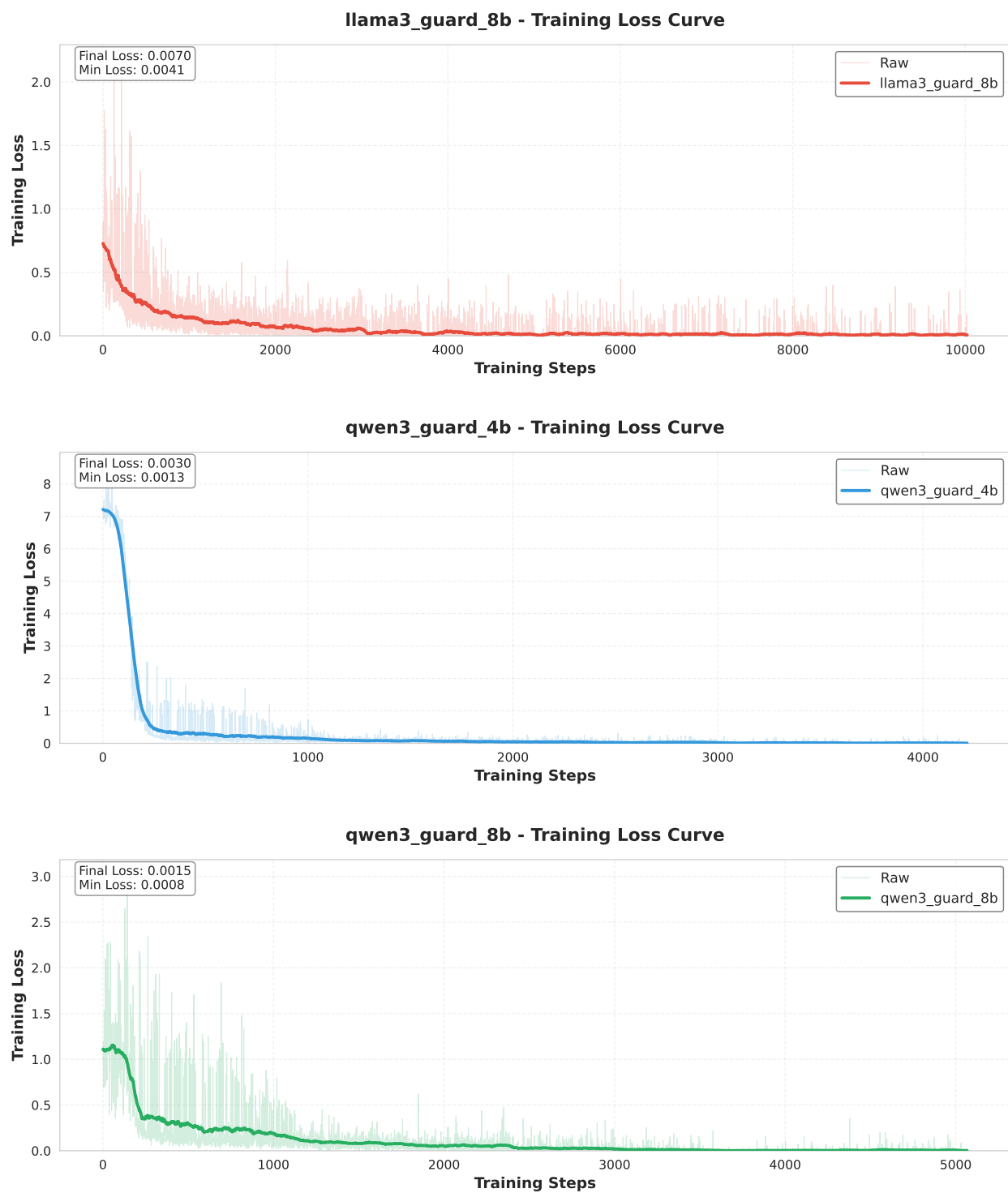


Figure 8 Training loss curves for BraveGuard-Llama-Guard-8B, BraveGuard-Qwen3-Guard-4B, and BraveGuard-Qwen3-Guard-8B. All three models show rapid early loss reduction followed by gradual convergence to a low-loss regime, indicating stable optimization under BraveGuard trajectory-level supervision.